

점프 투 자바

박응용 지음(위키독스 운영자)

이지스퍼블리싱

05

객체 지향 프로그래밍



05-1 객체 지향 프로그래밍이란?

05-2 클래스

05-3 메서드 더 살펴보기

05-4 값에 의한 호출과 객체에 의한 호출

05-1 객체 지향 프로그래밍이란?

- Calculator 클래스로 만든 별개의 계산기 cal1, cal2(객체)가 각각의 역할을 수행하는 예
  - 객체를 사용하면 계산기 수가 늘어나더라도 객체를 생성만 하면 됨

Do it! 코딩해 보세요

```
class Calculator {
    int result = 0;

    int add(int num) {
        result += num;
        return result;
    }
}

public class Sample {
    public static void main(String[] args) {
        Calculator cal1 = new Calculator();  // 계산기1 객체를 생성한다.
        Calculator cal2 = new Calculator();  // 계산기2 객체를 생성한다.
        System.out.println(cal1.add(3));
        System.out.println(cal1.add(4));

        System.out.println(cal2.add(3));
        System.out.println(cal2.add(7));
    }
}
```

실행 결과

```
3
7
3
10
```

05-2 클래스

클래스와 객체

- 클래스의 중요한 기능 – 객체(object)를 만드는 기능

Do it! 코딩해 보세요

```
class Animal {
}

public class Sample {
    public static void main(String[] args) {
        Animal cat = new Animal();
    }
}
```

  - new – 객체를 생성할 때 사용하는 키워드
  - Animal 클래스의 인스턴스(instance)인 cat, 즉 Animal의 객체가 만들어짐

05-2 클래스

클래스와 객체

- 과자 모양을 찍어 내는 과자 틀 = 클래스
- 과자 틀에 의해 만들어진 과자들 = 객체



- 무수히 많은 동물 객체(여기서는 cat, dog, horse, ...)들을 Animal 클래스로 만드는 예

```
Animal cat = new Animal();
Animal dog = new Animal();
Animal horse = new Animal();
...
```



05-2 클래스

▪ 객체 변수(instance variable)란?

- 클래스에 선언된 변수
- 도트 연산자(.)를 이용하여 접근

```
객체.객체변수 = 값
```

Do it! 코딩해 보세요

```
class Animal {
    String name;
}

public class Sample {
    public static void main(String[] args) {
        Animal cat = new Animal();
        System.out.println(cat.name);
    }
}
```

실행 결과

```
null
```

- null은 값이 할당되어 있지 않은 상태

05-2 클래스

메서드(method)란?

- 클래스 내에 구현된 함수
- 객체.메서드로 호출

Do it! 코딩해 보세요

```
class Animal {
    String name;

    public void setName(String name) {
        this.name = name;
    }
}

public class Sample {
    public static void main(String[] args) {
        Animal cat = new Animal();
        System.out.println(cat.name);
    }
}
```

- Animal 클래스의 객체 변수인 name에 값을 대입하고 setName 메서드 추가

- setName 메서드의 형태
  - 입력: String name
  - 출력: void ← '리턴값 없음'을 의미

05-2 클래스

▪ 메서드(method)란?

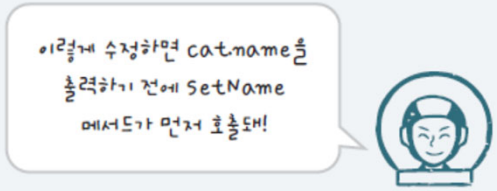
- setName 메서드를 호출할 수 있도록 main 메서드를 다음과 같이 수정

Do it! 코딩해 보세요

```
class Animal {
    String name;

    public void setName(String name) {
        this.name = name;
    }
}

public class Sample {
    public static void main(String[] args) {
        Animal cat = new Animal();
        cat.setName("boby");  // 메서드 호출
        System.out.println(cat.name);
    }
}
```



실행 결과

```
boby
```

05-2 클래스

- **객체 변수는 공유되지 않는다**
  - 객체 변수의 값은 독립적으로 유지된다
    = 클래스 존재의 이유
  - 객체 지향적(object oriented)
    - 객체 변수의 값이 독립적으로 유지되기 때문에 가능한 것

Do it! 코딩해 보세요

```
class Animal {
   String name;

   public void setName(String name) {
      this.name = name;
   }
}

public class Sample {
   public static void main(String[] args) {
      Animal cat = new Animal();
      cat.setName("boby");  // 메서드 호출

      Animal dog = new Animal();
      dog.setName("happy");

      System.out.println(cat.name);
      System.out.println(dog.name);
   }
}
```

실행 결과

```
boby
happy
```

05-3 메서드 더 살펴보기

메서드를 사용하는 이유

- 똑같은 내용을 반복해서 적는 대신 '어떤 입력값을 주었을 때 어떤 리턴값을 돌려준다'라는 메서드를 작성하는 것이 현명한 방법

- 메서드 정의의 예

> "sum 메서드는 두 개의 입력값(int 자료형인 a, int 자료형인 b)을 받으며,
> 리턴값은 두 개의 입력값을 더한 값(int 자료형)이다."

  - return은 메서드의 결괏값을 돌려주는 명령어

05-3 메서드 더 살펴보기

▪ 메서드를 사용하는 이유

▪ 메서드 정의의 예

Do it! 코딩해 보세요

```
public class Sample {
    int sum(int a, int b) {
        return a + b;
    }

    public static void main(String[] args) {
        int a = 3;
        int b = 4;

        Sample sample = new Sample();
        int c = sample.sum(a, b);

        System.out.println(c);
    }
}
```

실행 결과

```
7
```

05-3 메서드 더 살펴보기

▪ 매개 변수와 인수

- 매개 변수(parameter) - 메서드에 전달된 입력값을 저장하는 변수
- 인수(arguments)는 메서드를 호출할 때 전달하는 입력값

Do it! 코딩해 보세요

```
public class Sample {
    int sum(int a, int b) {  // a, b 는 매개 변수
        return a+b;
    }

    public static void main(String[] args) {
        Sample sample = new Sample();
        int c = sample.sum(3, 4);  // 3, 4는 인수

        System.out.println(c);
    }
}
```



05-3 메서드 더 살펴보기

메서드의 입력값과 리턴값

메서드의 구조

- 메서드는 입력값을 가지고 어떤 처리를 하여 적절한 리턴값을 돌려주는 블랙박스
- 리턴 자료형 – 메서드 수행 후 리턴되는 값의 자료형

```
리턴자료형 메서드명(입력자료형1 매개변수1, 입력자료형2 매개변수2, …) {
    …
    return 리턴값;  // 리턴 자료형이 void인 경우에는 return 문이 필요 없다.
}
```

05-3 메서드 더 살펴보기

메서드의 입력값과 리턴값

입력값과 리턴값이 모두 있는 메서드

```
int sum(int a, int b) {
    return a+b;
}
```

- 호출 방법

```
리턴값 받을 변수 = 객체.메서드명(입력인수1, 입력인수2, …)
```

- sum 메서드 사용 예

```
Sample sample = new Sample();
int result = sample.sum(3, 4);
```

05-3 메서드 더 살펴보기

메서드의 입력값과 리턴값

입력값이 없는 메서드

```
String say() {
    return "Hi";
}
```

- 호출 방법

```
리턴값 받을 변수 = 객체.메서드명()
```

- say 메서드 사용 예

Do it! 코딩해 보세요

```
public class Sample {
    String say() {
        return "Hi";
    }

    public static void main(String[] args) {
        Sample sample = new Sample();
        String a = sample.say();
        System.out.println(a);  // "Hi" 출력
    }
}
```

05-3 메서드 더 살펴보기

메서드의 입력값과 리턴값

리턴값이 없는 메서드

```
void sum(int a, int b) {
    System.out.println(a+"과 "+b+"의 합은 "+(a+b)+"입니다.");
}
```

- 리턴 자료형 부분에 void라고 표기
- 호출 방법

```
객체.메서드명(입력인수1, 입력인수2, …)
```

- 사용 예

Do it! 코딩해 보세요

```
public class Sample {
    void sum(int a, int b) {
        System.out.println(a+"과 "+b+"의 합은 "+(a+b)+"입니다.");
    }

    public static void main(String[] args) {
        Sample sample = new Sample();
        sample.sum(3, 4);
    }
}
```

실행 결과

```
3과 4의 합은 7입니다.
```

- System.out.println 문은 메서드 내에서 사용되는 문장일 뿐 리턴값은 없음
- 리턴값은 오직 return 명령어로만 반환 가능

05-3 메서드 더 살펴보기

▪ 메서드의 입력값과 리턴값

▪ 입력값과 리턴값이 없는 메서드

```
void say() {
    System.out.println("Hi");
}
```

- 호출 방법

```
객체.메서드명()
```

- 사용 예 →

Do it! 코딩해 보세요

```
public class Sample {
    void say() {
        System.out.println("Hi");
    }

    public static void main(String[] args) {
        Sample sample = new Sample();
        sample.say();
    }
}
```

05-3 메서드 더 살펴보기

return의 또 다른 쓰임

- 메서드를 빠져나가고 싶다면 return을 단독으로 사용

Do it! 코딩해 보세요

```
public class Sample {
    void sayNick(String nick) {
        if ("바보".equals(nick)) {
            return;
        }
        System.out.println("나의 별명은 "+nick+" 입니다.");
    }

    public static void main(String[] args) {
        Sample sample = new Sample();
        sample.sayNick("야호");
        sample.sayNick("바보");  // 출력되지 않는다.
    }
}
```

- sayNick 메서드는 입력으로 받은 문자열을 포함한 문장을 출력
- 여기서 쓰인 return은 메서드를 호출한 곳에 리턴값을 돌려주는 역할이 아니라 메서드를 즉시 빠져나가는 역할
  - 문자열을 출력한다는 것과 리턴값이 있다는 것은 전혀 다른 말

05-3 메서드 더 살펴보기

▪ 메서드 내에서 선언된 변수의 효력 범위

- 메서드 안에서 사용하는 변수의 이름을 메서드 밖에서 사용한 이름과 동일하게 사용한 경우

Do it! 코딩해 보세요

```
public class Sample {
    void varTest(int a) {
        a++;
    }

    public static void main(String[] args) {
        int a = 1;
        Sample sample = new Sample();
        sample.varTest(a);
        System.out.println(a);
    }
}
```

- varTest 메서드는 입력으로 들어온 int 자료형의 값을 1만큼 증가시키는 역할

- 결괏값이 2가 아닌 1이 출력되는 이유
  - 메서드에서 사용한 매개 변수 a는 메서드 안에서만 쓰이는 변수이기 때문
  - 메서드에서 쓰이는 매개 변수의 이름과 메서드 밖의 변수 이름이 같더라도 서로 전혀 영향 주지 않음

```
void varTest(int b) {
    b++;
}
```

05-3 메서드 더 살펴보기

▪ 메서드 내에서 선언된 변수의 효력 범위

- varTest라는 메서드를 이용해 main 메서드 외부의 a의 값을 1만큼 증가시킬 수 있는 방법

Do it! 코딩해 보세요

```
public class Sample {
    int varTest(int a) {
        a++;
        return a;
    }

    public static void main(String[] args) {
        int a = 1;
        Sample sample = new Sample();
        a = sample.varTest(a);
        System.out.println(a);
    }
}
```

실행 결과

```
2
```

- varTest 메서드에 return 문을 이용
- varTest 메서드는 입력으로 들어온 값을 1만큼 증가시켜 리턴

05-3 메서드 더 살펴보기

▪ 메서드 내에서 선언된 변수의 효력 범위

▪ varTest 입력값이 int 자료형이 아닌 객체인 경우

Do it! 코딩해 보세요

```
public class Sample {

    int a;  // 객체 변수 a

    void varTest(Sample sample) {
        sample.a++;
    }

    public static void main(String[] args) {
        Sample sample = new Sample();
        sample.a = 1;
        sample.varTest(sample);
        System.out.println(sample.a);
    }
}
```

실행 결과

```
2
```

- sample 객체의 객체 변수 a의 값이 원래는 1이었으나 varTest 메서드 실행 후 1만큼 증가되어 2가 출력되는 것을 확인

- varTest 메서드의 입력 파라미터가 값이 아닌 Sample 클래스의 객체
  → 입력으로 전달받은 sample 객체의 객체 변수 a의 값이 증가

05-4 값에 의한 호출과 객체에 의한 호출

▪ 메서드 내에서 선언된 변수의 효력 범위

- 메서드에 객체를 전달할 경우 메서드에서 객체 변수의 값을 변경할 수 있음
  - update 메서드는 전달받은 숫자를 1 증가시킴
- 메서드의 입력으로 객체를 전달하면 메서드가 입력받은 객체를 그대로 사용하기 때문에 객체의 속성값을 변경하면 메서드 수행 후에도 객체의 변경된 속성값이 유지됨

Do it! 코딩해 보세요

```
class Updater {
    void update(Counter counter) {
        counter.count++;
    }
}

class Counter {
    int count = 0;  // 객체 변수
}

public class Sample {
    public static void main(String[] args) {
        Counter myCounter = new Counter();
        System.out.println("before update:"+myCounter.count);
        Updater myUpdater = new Updater();
        myUpdater.update(myCounter);
        System.out.println("after update:"+myCounter.count);
    }
}
```

실행 결과

```
before update:0
after update:1
```



감사합니다.